ICTCE04

International College
Rangsit University
52/347 Muang-Ake, Phaholyothin Road, Lak-Hok, Muang, Pathumthani 12000, Thailand
T. (662) 997-2200-30 Ext. 4012-4020, F. (662) 997-2200-30 Ext. 4016
www.rsuip.org

Cooperative Education Liaison:
Miss Srinuan Ketkaew
T. (662) 997-2222 Ext. 4012 E-mail: srinuan@rsu.ac.th

Weekly Progress Report Form

Name - Mr. Sai Swam Wan Hline, Student ID - 6609216

Tel. 0649435041, E-mail - saiswamwan.h66@rsu.ac.th,

Company Name /Employer Name - **HTZKKL Technologies Co.,Ltd.**

Reporting Period: *Week No.5, October 2026, 30/9/2026 - 6/10/2026, Total 40 Hours*

1. Name of project/task/issue/topic of student's responsibility.

(List each of the performed tasks.)

Project: Multi-Tenant School Intranet & Academic Management Platform

Task 1: Backend dependency management migration to uv and standardized pyproject.toml packaging.

Task 2: Full-stack persistence layer migration to Async SQLAlchemy 2.0 with `asyncpg` (PostgreSQL) and `aiosqlite` (SQLite).

Task 3: Asynchronous refactoring across all 16 feature modules (services, routers, authentication guards, and permissions).

Task 4: Resolution of asynchronous ORM lazy-loading pitfalls (MissingGreenlet) and database connection pooling configuration.

Task 5: Resolution of `asyncpg` timezone serialization edge cases, dead code elimination, and asynchronous test suite upgrades (`httpx.AsyncClient`).

2. Summary of the performed tasks.

(Describe the type of work that the student has accomplished.)

During the fifth week of the project:

1.Modern Dependency Tooling (uv): Replaced legacy pip and flat requirements.txt with Astral's uv package manager and pyproject.toml with deterministic lockfiles

(uv.lock), accelerating CI dependency resolution and build speeds.

2.Async Persistence Architecture: Upgraded the core database engine to asynchronous SQLAlchemy 2.0. Integrated high-performance native async drivers (`asyncpg` for PostgreSQL in production and `aiosqlite` with greenlet for local testing/development), deprecating blocking psycopg2-binary.

3.Comprehensive Async Service Refactor: Systematically migrated all 16 backend modules (auth, students, teachers, academics, attendance, classes, timetable, announcements, parents, schools, analytics, examinations, finance, promotions, telegram, viber) from synchronous database sessions to AsyncSession and explicit 2.0 query syntax (select(), execute(), scalars()).

4.Relationship & Concurrency Hardening:

Configured explicit eager loading (selectinload / joinedload) on critical relationships across models to eliminate asynchronous MissingGreenlet exceptions caused by detached lazy-loading.

Guarded Alembic migrations (run_migrations_online()) to execute safely inside active asyncio event loops.

Refactored the financial document counter engine with asynchronous row-level locking (SELECT ... FOR UPDATE) to guarantee collision-free invoice numbering under heavy concurrency.

5.Driver Fixes & Test Suite Overhaul:

Fixed asyncpg timezone serialization mismatches during user login token generation.

Pruned orphaned helper utilities, redundant modals, and dead scaffolding code.

Upgraded the automated test framework to utilize httpx.AsyncClient (DualClient) with isolated asynchronous sessions per test case.

3. Application of concepts, theories, knowledge, or experiences that applied to cooperative education.

Asynchronous Programming & Non-Blocking I/O: Applied Python `asyncio` coroutines, event loop lifecycles, and cooperative multitasking principles to eliminate thread-blocking database queries, drastically increasing concurrent throughput.

SQLAlchemy 2.0 Paradigm & Modern ORM Design: Transitioned from legacy query builder patterns (session.query()) to modern functional statement expressions (select(), update(), delete()), enforcing strong typing and compile-time correctness.

Connection Pooling & Resource Management: Designed optimized pool strategies, configuring NullPool for embedded SQLite environments and tuned connection pool recycling for PostgreSQL to prevent connection exhaustion.

Modern Build Systems & Packaging Standards: Implemented PEP 518/PEP 621 packaging specifications with pyproject.toml and deterministic dependency resolution.

Integration Testing with Asynchronous Clients: Utilized httpx.AsyncClient and transactional rollbacks to verify asynchronous web endpoints under production-like conditions.

4. Problems/obstacles encountered during the work.

(Provide details on how to solve the problems/obstacles.)

Problem 1: MissingGreenlet Exceptions on Relational Attribute Access:

Detail: In asynchronous SQLAlchemy, accessing related models (e.g. student.enrollments or user.teacher) that were not eagerly loaded triggered sqlalchemy.exc.MissingGreenlet: greenlet_spawn has not been called because lazy-loading cannot execute I/O implicitly in async mode.

Solution: Audited all ORM model queries across services and explicitly applied eager loading via selectinload() or joinedload(), and converted computed properties into explicit service queries.

Problem 2: `asyncpg` Timezone-Aware vs. Naive Datetime Rejection:

Detail: PostgreSQL via `asyncpg` strictly rejects passing timezone-naive datetime objects into timezone-aware TIMESTAMPTZ database columns during user authentication updates (last_login_at).

Solution: Standardized all server timestamp generations to explicit UTC timezone-aware objects (datetime.now(timezone.utc)), ensuring seamless serialization across all database engines.



(……Nay Min.....)

Job Supervisor

Date ……6/10/2026.....



(….Sai Swam Wan Hline.....)

Student

Date ……6/10/2026.....